

PIKA BOY COLOR

CSS Art Festival

Pikachu Interactive Console Development Report

This report outlines the design, technical structure, and debugging workflows involved in constructing the Interactive CSS Pikachu mascot and GameBoy console. The project was hand-coded and developed from scratch in accordance with the strict rules of the CSS Art Festival: using only HTML and CSS, with zero SVGs, canvas, image tags, or background-image URLs.

Core Implementation Stack

- **100% Pure HTML/CSS:** Constructed using standard layered div and span elements styled with custom border-radius properties, box-shadows, and rotational/skew transforms.
 - **Interactive CSS Engine:** Utilizes the CSS Checkbox Hack (sibling selection combinators ~ linked to hidden input elements) to change states and trigger styles dynamically without a single line of JavaScript.
 - **Dynamic Environment:** Houses an LCD screen backdrop with sliding hills, animated drifting clouds, and a Day/Night cycle (sun setting, moon rising, stars twinkling, and glowing cheeks).
 - **Four Operational Modes:** Supports Power toggling, Day/Night cycles, B-button Sleep mode (eyes close, breathing slows, Zzz particles float), and A-button Shock mode (screen shakes, sparks flash, shockwaves expand, HP drains).
-

Pikachu Geometry & Code Architecture

Design Decisions and Layer Construction

To deliver a premium, high-fidelity aesthetic, Pikachu's geometry is composed of multiple layers styled using organic percentages for curves. The following structures break down how the mascot was constructed:

1. Pikachu Head & Face Details

The head is an organic sphere built with border-radius: 50% 50% 43% 43% / 58% 58% 42% 42%. The eyes contain dual-pupil white highlights at diagonal angles to represent gloss. The cheeks glow in the dark via transition-driven box-shadows (inset for base, outer glow for night/shock modes). The mouth contains overlapping curved lip lines styled using borders, hiding a scale-animated mouth opening with a tongue element.

2. Zigzag Lightning-Bolt Tail

Rather than using an SVG or image, the lightning tail is made of 5 distinct joints (nested divs) styled with absolute positioning and transforms. The base joint is brown (stripe color), while the following joints are yellow, with the tip joint rotated at 35deg and skewed by -20deg (skewX) to create the characteristic bolt arrowhead.

3. Interactive CSS State Machine

Toggles are driven by standard HTML checkbox inputs located inside the app container, followed by the console-wrapper. When buttons (labels) are clicked, they check/uncheck the inputs. In CSS, general sibling selectors map the checked state to visual styles:

- #power-toggle:checked ~ .console-wrapper .screen-display { filter: brightness(1); }
 - #sleep-toggle:checked ~ .console-wrapper .sleep-eyelid { height: 100%; }
 - #shock-toggle:checked ~ .console-wrapper .screen-display { animation: screen-shake 0.15s; }
-

Debugging & Optimization Log

Resolving Critical Implementation Challenges

During development, three critical challenges were identified and optimized to ensure usability, aesthetic compliance, and high performance:

Bug 1: Sibling Selector Nesting (Black Screen Fixed)

- Observation: The power-off screen remained completely black, and clicking the power switch did not turn the console on.
- Analysis: The checkbox controllers were placed outside the app-container, but selectors targeted "~ .console-wrapper". Because the console wrapper was nested inside the app container, the sibling relation failed.
- Resolution: Moved the inputs inside the ".app-container" directly preceding the console. Restructured selectors to target "~ .console-wrapper", which is now a direct subsequent sibling. The screen now boots instantly by default.

Bug 2: Browser State Caching (autocomplete="off")

- Observation: If a user turned off the console and refreshed, the screen remained black on reload.
- Analysis: Browsers automatically cache checkbox checked states across soft refreshes, preserving the unpowered state.
- Resolution: Added autocomplete="off" to all checkbox state controllers. The console is now guaranteed to load in its default powered-on, animated state on every page load.

Bug 3: Small Power Button Click Targets (UX Enlarged)

- Observation: The power switch slot was small (32x14px), making it difficult to click on mobile or high-DPI displays.
 - Analysis: Power switches must be accessible and easy to toggle.
 - Resolution: Wrapped both the word "POWER" and the switch slot inside a single, large clickable <label> tag and scaled the switch box and knob up. Clicking the label text or the knob now toggles the power slider.
-

